
Kết luận trước, lý lẽ sau

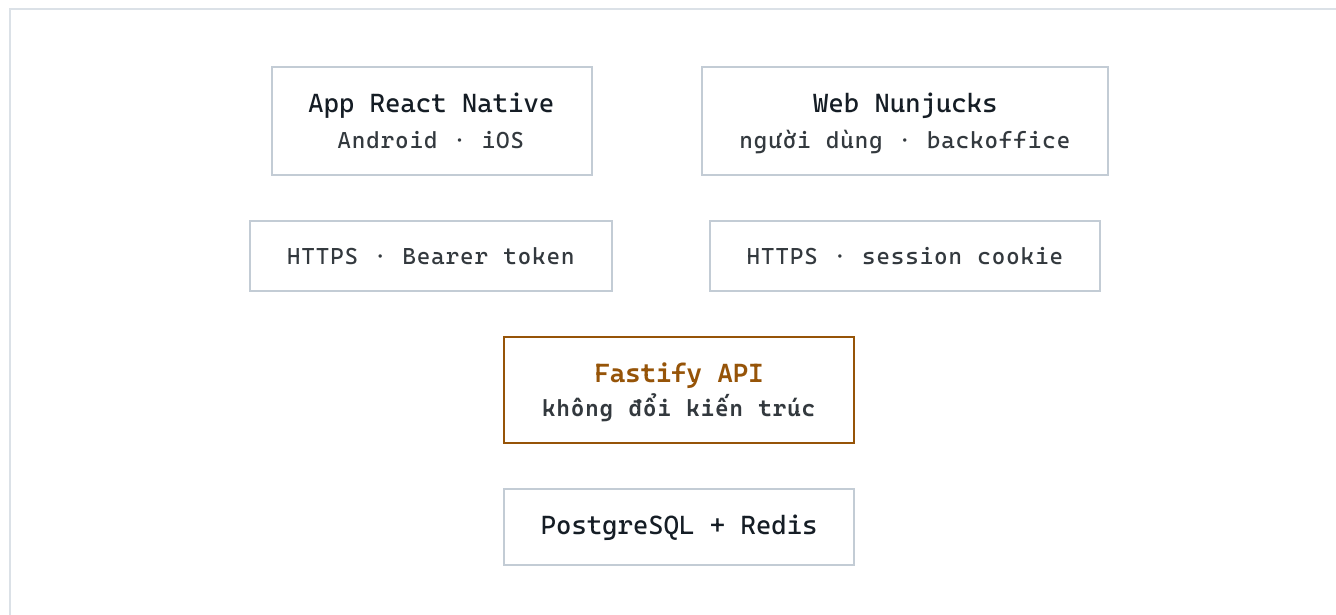
React Native + Expo + EAS Build là hướng đúng cho đề bài này. App là project riêng, giao tiếp với backend qua HTTPS, không phụ thuộc giao diện web. Một codebase ra được cả hai nền tảng.

Máy bạn **đã đủ điều kiện gần như ngay lập tức**: vì build chạy trên hạ tầng của Expo và bạn test bằng máy thật, bạn không cần cài Android Studio, không cần JDK, không cần giả lập. Chỉ thêm một lệnh `npm i -g eas-cli` là xong môi trường.

Điểm cần quyết sớm: **iOS không cài trực tiếp được như Android**. Muốn thử app trên iPhone thật vượt quá phạm vi Expo Go thì phải có tài khoản Apple Developer ngay từ khâu kiểm thử, chứ không phải đợi đến lúc phát hành.

Kiến trúc

App mới đứng cạnh web hiện tại, không thay thế. Cả hai dùng chung một backend, một cơ sở dữ liệu.

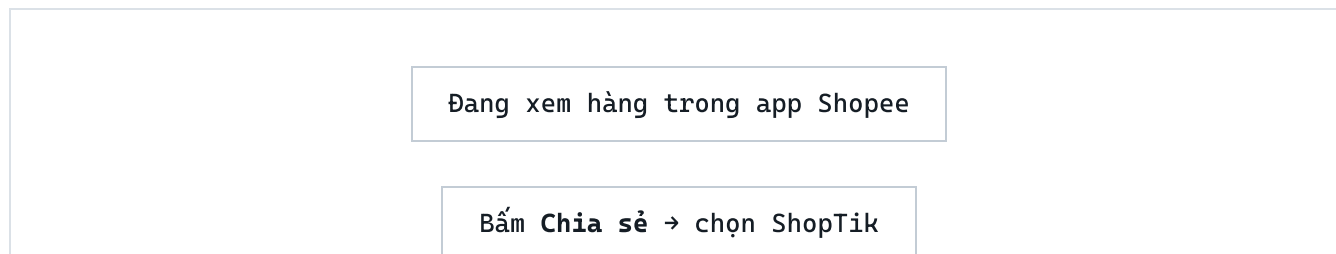


Khu vận hành `/backoffice` giữ nguyên trên web. Không có lý do làm app quản trị — admin làm việc trên máy tính, và mọi màn hình đó đều là bảng dữ liệu dày.

Điều phải thiết kế lại, không bê nguyên từ web

Web mở đầu bằng ô dán link sản phẩm. Trên điện thoại, hành vi đó gần như không xảy ra.

Người dùng đang ở *trong* app Shopee hoặc TikTok Shop khi họ nhìn thấy món hàng. Bắt họ thoát ra, mở app khác, dán link — là đánh mất phần lớn lượt mua. Luồng đúng trên di động là chiều ngược lại:



App bật lên, hiện ngay tiền hoàn

Bấm Mua → quay lại Shopee qua link Affiliate

iOS gọi cái này là Share Extension, Android là intent filter ACTION_SEND . Ô dán link thủ công vẫn giữ làm phương án dự phòng, nhưng không còn là lối vào chính.

Đừng trông vào đọc clipboard tự động. Từ iOS 14, mỗi lần app đọc clipboard hệ điều hành đều hiện cảnh báo cho người dùng thấy. Thiết kế xoay quanh tính năng đó sẽ vừa phiền vừa mất lòng tin.

Phạm vi phiên bản 1

Khu người dùng trên web hiện có 20 route. Không cần chuyển hết. Sáu màn dưới đây đủ để app hoàn thành trọn vẹn vòng đời kiếm tiền của người dùng.

MÀN HÌNH	VIỆC NGƯỜI DÙNG LÀM	PHIÊN BẢN
Đăng nhập / Đăng ký	Tạo tài khoản, xác minh email, quên mật khẩu	V1
Trang chủ	Nhận link chia sẻ → xem tiền hoàn → bấm Mua	V1
Đơn hàng	Theo dõi đơn, trạng thái đối soát, báo đơn thiếu	V1
Ví	Xem số dư chờ và khả dụng, lịch sử biến động	V1
Rút tiền	Thêm tài khoản ngân hàng, tạo lệnh rút, xem lịch sử	V1
Tài khoản	Đổi hồ sơ, đăng xuất thiết bị, xóa tài khoản	V1
Nhiệm vụ · Giới thiệu	Mốc thưởng, mời bạn bè	V2
Khám phá · Link đã tạo	Duyệt nội dung, xem lại link cũ	V2

Chat hỗ trợ · Thông báo	Nhấn CSKH, đọc thông báo trong app	v2
-------------------------	------------------------------------	----

Xóa tài khoản nằm ở v1 không phải vì tính năng hay, mà vì **cả Apple lẫn Google đều bắt buộc** app có tài khoản phải cho người dùng tự xóa ngay trong app. Thiếu là bị trả hồ sơ.

Bộ công cụ

Chọn theo tiêu chí: chạy được trên Windows, không cần native code viết tay ở v1, và giữ TypeScript xuyên suốt để dùng chung kiểu dữ liệu với backend.

VAI TRÒ	CHỌN	VÌ SAO
Nền tảng	<code>react-native + expo</code>	Một codebase, hai nền tảng, TypeScript như backend
Điều hướng	<code>expo-router</code>	Định tuyến theo file, hợp với deep link từ share sheet
Gọi API	<code>@tanstack/react-query</code>	Cache, retry, refetch — hợp với dữ liệu ví và đơn hàng
Lưu token	<code>expo-secure-store</code>	Keychain trên iOS, Keystore trên Android
Mở link Affiliate	<code>expo-web-browser</code>	Quan trọng nhất — xem mục Rủi ro bên dưới
Nhận link chia sẻ	<code>expo-share-intent</code>	Share Extension iOS + intent filter Android
Thông báo đẩy	<code>expo-notifications</code>	Gói chung APNs và FCM sau một API
Build	<code>eas build</code>	Build iOS trên máy Mac trong cloud của Expo
Nộp store	<code>eas submit</code>	Đẩy thẳng lên Play Console và App Store Connect

Và nóng	eas update	Sửa JS không cần chờ store duyệt lại
---------	------------	--------------------------------------

eas update đáng chú ý riêng với nghiệp vụ này: chính sách hoàn tiền, tỷ lệ chia, văn bản điều khoản đều là thứ hay phải sửa gấp. Có kênh vá nóng nghĩa là sửa trong vài phút thay vì chờ Apple duyệt vài ngày.

Môi trường trên máy này

Đã kiểm tra máy bạn: Ryzen 5 5600, 32 GB RAM, 162 GB trống ổ F. Thừa sức, và thực ra còn cần ít hơn bạn nghĩ.

THÀNH PHẦN	HIỆN TRẠNG	CÓ CẦN KHÔNG
Node.js 24.18	ĐÃ CÓ	Có
Git 2.55	ĐÃ CÓ	Có
VS Code	ĐÃ CÓ	Có
eas-cli	CHƯA CÓ	Có — npm i -g eas-cli
Tài khoản Expo	CHƯA CÓ	Có — miễn phí
Android Studio	CHƯA CÓ	Không cần — chỉ dùng cho giả lập
JDK 17	CHƯA CÓ	Không cần — EAS build trên cloud
Điện thoại Android thật	TỰ CHUẨN BỊ	Có
iPhone thật	TỰ CHUẨN BỊ	Có — Windows không chạy được giả lập iOS

Vì bạn đã chọn test bằng máy thật, toàn bộ khối nặng nhất của môi trường Android (Android Studio ~10 GB, SDK, JDK, giả lập) đều bỏ được. Đây là khác biệt lớn so với quy trình React Native truyền thống.

Cài lên máy thật: hai nền tảng rất khác nhau

Đây là chỗ dễ vỡ kế hoạch nhất. Android dễ đến mức tưởng iOS cũng vậy — không phải.

Android	iOS
- Build hồ sơ preview ra file .apk - Tải file về, gửi qua Zalo hay USB, bấm cài - Chỉ cần bật "cài từ nguồn không xác định" Không cần tài khoản nào cả - Gửi cho người khác test thoải mái	- Không có chuyện cài thẳng .ipa như Android - Phải qua TestFlight, hoặc bản ad-hoc có đăng ký sẵn mã máy - Cả hai đường đều cần Apple Developer Account - Đường miễn phí duy nhất là Expo Go — nhưng chỉ chạy được phần JavaScript - Cần share extension hay push là hết cửa miễn phí

Hệ quả về ngân sách: khoản Apple Developer 99 USD/năm phải chi *ngay từ giai đoạn kiểm thử iOS*, không phải đợi lúc nộp App Store. Nếu muốn hoãn khoản này, hãy làm xong và test kỹ Android trước, iOS đi sau một nhịp.

KHOẢN CHI	GIÁ	Kiểu trả	BẮT BUỘC KHI
Google Play Console	~25 USD	Một lần, trọn đời	Lúc nộp CH Play
Apple Developer Program	~99 USD	Mỗi năm	Ngay từ lúc test trên iPhone
EAS Build	0 USD	Gói miễn phí có xếp hàng chờ	Trả phí khi cần build nhanh

Giá hai store ổn định nhiều năm nhưng nên xác nhận lại lúc đăng ký. Giá EAS thì Expo đổi khá thường xuyên.

Backend cần bổ sung gì

Tin tốt: luồng kiểm tiền cốt lõi đã có API đầy đủ. Phần thiếu nằm ở nhánh ví và tài khoản. Bảng này đối chiếu từng màn hình v1 với API thực tế trong `src/routes/api/`.

APP CẦN	API HIỆN TẠI	TRẠNG THÁI
Đăng nhập, đăng ký, quên mật khẩu, xác minh email	<code>/api/v1/auth/*</code>	ĐỦ DÙNG
Hồ sơ người dùng	<code>/api/v1/me</code>	ĐỦ DÙNG
Xem tiền hoàn từ link	<code>/api/v1/products/preview</code>	ĐỦ DÙNG
Tạo link mua	<code>/api/v1/products/purchase</code>	ĐỦ DÙNG
Danh sách đơn hàng	<code>/api/v1/me/orders</code>	ĐỦ DÙNG
Số dư ví và lịch sử	<code>/api/v1/me/wallet</code>	ĐỦ DÙNG
Lịch sử rút tiền	<code>/api/v1/me/withdrawals</code>	ĐỦ DÙNG
Báo đơn chưa ghi nhận	<code>/api/v1/support/missing-order</code>	ĐỦ DÙNG
Đăng nhập bằng token	chỉ có session cookie	PHẢI LÀM
Tạo lệnh rút tiền	chỉ có form web	PHẢI LÀM
Tài khoản ngân hàng	chỉ có form web	PHẢI LÀM
Xóa tài khoản	chưa có ở phía người dùng	PHẢI LÀM
Đổi hồ sơ, đăng xuất mọi thiết bị	chỉ có form web	PHẢI LÀM
Lưu token thiết bị để đẩy thông báo	chưa có	V2
Nhiệm vụ, giới thiệu, khám phá, chat	chỉ có form web	V2

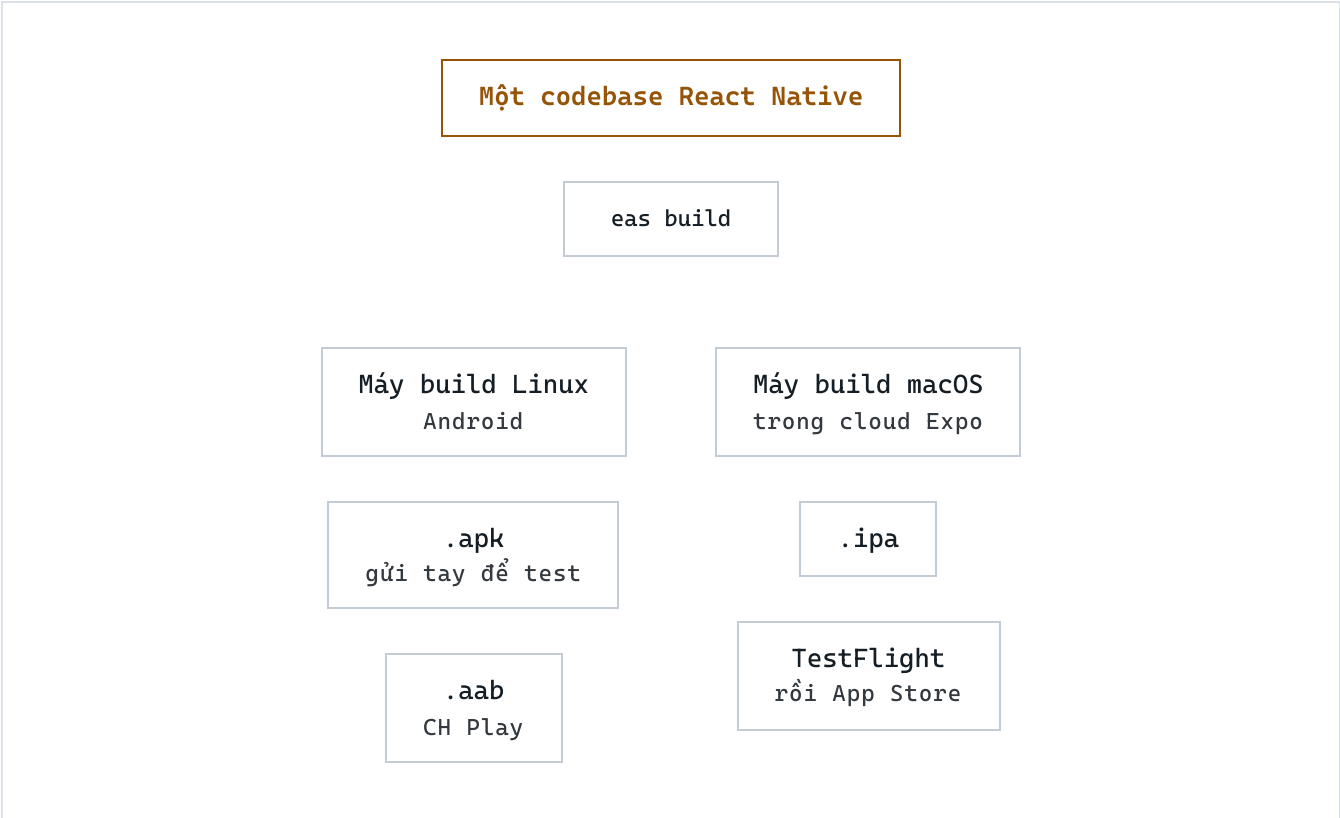
Vì sao phải thêm token, không dùng lại cookie

Backend đang xác thực bằng cookie `aff_session` với `sameSite: lax`, cộng thêm CSRF kiểu `double-submit`. Cơ chế này thiết kế cho trình duyệt cùng nguồn. App di động gọi từ nguồn khác nên cookie sẽ không được gửi kèm.

Cách sạch là cấp `access token` và `refresh token` khi đăng nhập, chấp nhận header `Authorization: Bearer`, và miễn kiểm tra CSRF cho các route dùng `bearer` — an toàn, vì trình duyệt không tự đính kèm `bearer token` như cách nó tự gửi cookie. Web hiện tại không bị ảnh hưởng gì, hai cơ chế chạy song song.

Từ code đến cửa hàng

Ba hồ sơ build trong `eas.json`, mỗi hồ sơ phục vụ một mục đích khác nhau.



HỒ SƠ	DÙNG KHI	RA FILE
development	Lập trình hằng ngày, có gắn dev client	.apk / .ipa nội bộ

preview	Gửi người khác dùng thử	.apk
production	Nộp cửa hàng	.aab / .ipa

Expo Go không thay thế được development build

Expo Go chỉ chạy được những native module Expo đóng gói sẵn. Đủ cho giai đoạn dựng giao diện và gọi API. Nhưng ngay khi bạn thêm share extension để nhận link từ app Shopee, Expo Go hết tác dụng và phải chuyển sang development build. Cứ dự trù bước đó từ đầu thay vì phát hiện giữa chừng.

Điều kiện của hai cửa hàng

APP STORE

- Xóa tài khoản ngay trong app — bắt buộc
- Tài khoản demo cho người duyệt, kèm hướng dẫn luồng hoàn tiền
- Khai báo quyền riêng tư chi tiết theo từng loại dữ liệu
- App phải là app thật, không phải website đóng gói — React Native đã thỏa
- Hoàn tiền cho hàng vật lý mua ngoài **không** tính là mua trong app

CH PLAY

- Xóa tài khoản trong app và một đường dẫn web tương ứng
- Biểu mẫu An toàn dữ liệu
- Đường dẫn chính sách quyền riêng tư — đã có /chinh-sach-nguoi-dung
- Đáp ứng mức API Android tối thiểu tại thời điểm nộp
- Khai báo đúng nhóm tài chính nếu có luồng rút tiền

Mô hình hoàn tiền không vướng chính sách của cả hai bên — ShopBack, Rakuten và nhiều app cùng loại đang có mặt trên cả hai cửa hàng. Rào cản thực tế là thủ tục, không phải mô hình kinh doanh.

Lộ trình

Năm giai đoạn, xếp theo thứ tự phụ thuộc. Giai đoạn 0 nên làm trước tiên vì nó xác nhận toàn bộ chuỗi công cụ trước khi bạn đầu tư công sức vào giao diện.

GĐ 0 Thông chuỗi công cụ

Mục tiêu duy nhất: thấy một app trắng chạy được trên điện thoại Android thật của bạn.

- Cài `eas-cli` , tạo tài khoản Expo
 - Khởi tạo project Expo có TypeScript và `expo-router`
 - Chạy `eas build --profile preview --platform android`
 - Tải `.apk` , cài lên máy, mở lên
-

GĐ 1 Mở đường cho app ở backend

Làm trên repo hiện tại. Không đụng gì tới web đang chạy.

- Cấp access token và refresh token, chấp nhận `Authorization: Bearer`
 - Đăng ký CORS cho nguồn của app
 - Bổ sung API: tạo lệnh rút tiền, tài khoản ngân hàng, đổi hồ sơ
 - Thêm xóa tài khoản tự phục vụ — chặn cứng của cả hai store
-

GĐ 2 Dựng sáu màn hình v1

Giai đoạn dài nhất về khối lượng, nhưng ít rủi ro kỹ thuật nhất.

- Đăng nhập, Trang chủ, Đơn hàng, Ví, Rút tiền, Tài khoản
 - Mở link Affiliate bằng `expo-web-browser` , tuyệt đối không dùng `webview` nhúng
 - Test quy kết bằng đơn hàng thật càng sớm càng tốt
-

GĐ 3 Thêm phần chi app mới làm được

Đây là chỗ app vượt hẳn web, và cũng là lý do người dùng giữ app trong máy.

- Nhận link chia sẻ từ app sàn — chuyển sang development build ở bước này
 - Thông báo đẩy khi đơn được duyệt và khi tiền sang ví khả dụng
 - Đăng nhập bằng vân tay hoặc khuôn mặt
-

GD 4

Nộp cửa hàng

Android trước, iOS sau — vòng duyệt của Google thường nhanh hơn và ít bị trả hồ sơ hơn.

- Chuẩn bị ảnh chụp màn hình, mô tả, biểu tượng, màn hình chờ
- Điền biểu mẫu quyền riêng tư của cả hai bên
- Tạo tài khoản demo cho người duyệt Apple
- `eas submit` cho từng nền tảng

Ba rủi ro cần canh

Quy kết đơn hàng dứt khi nhảy sang app sàn

Đây là rủi ro nghiêm trọng nhất, và nó thuộc về nghiệp vụ chứ không phải lựa chọn công nghệ. Khi mở link Affiliate, phải dùng trình duyệt hệ thống (SFSafariViewController trên iOS, Chrome Custom Tabs trên Android) để hệ điều hành chuyển giao đúng sang app Shopee và giữ nguyên tham số Sub ID. Nếu mở bằng webview nhúng, lượt chuyển đổi có thể mất và nhiều mạng Affiliate còn cấm hẳn kiểu này. Hãy kiểm chứng bằng đơn thật ở Giai đoạn 2, trước khi đổ công vào giao diện.

Chi phí Apple đến sớm hơn dự tính

Nhiều kế hoạch giả định chỉ tốn 99 USD lúc phát hành. Thực tế bạn cần nó ngay khi muốn thử bản build thật trên iPhone. Nếu ngân sách cần giãn, làm trọn Android trước rồi iOS theo sau.

Xếp hàng ở gói EAS miễn phí

Gói miễn phí giới hạn số lượt build và phải chờ đến lượt. Giai đoạn đầu không sao, nhưng gần ngày nộp store — lúc bạn build lại liên tục để sửa lỗi — thời gian chờ sẽ thành nút cổ chai. Dự trù nâng gói trong tháng phát hành.